

Rakib SHEIKH
Travaux pratiques : Big Data
rakib.sheikh@cyu.fr

TP 2 : Prise en main du langage Scala

Pré-requis :

- Java (via sdkman si possible) <https://sdkman.io/> (pré-requis pour la partie 2)
 - Pour ceux qui sont sous Windows : Java 1.8 : <https://adoptium.net/temurin/releases/?os=windows&arch=x64&package=jdk&version=8>
- Pour les linux et mac : Installez la dernière version de sbt, et la version 2.12.15 de Scala via sdkman
- Pour Windows... je n'utilise pas Windows donc téléchargez Scala à l'aide de la doc officiel et faite moi un retour si cela fonctionne..
- Idéalement IntelliJ avec le plugin Big Data

Nous supposons que vous êtes sur un Unix, à vous de trouver les commandes pour Windows pour ceux qui ne sont pas sur Unix.

Le TP est divisé en deux parties :

- La partie introductive aidant à la transition Java -> Scala.
- Un Handson sous la forme de test unitaire pour prendre en main directement vos acquis.

Si vous n'êtes pas à l'aise avec Java, vous pouvez passer directement à la partie Handons.

Partie 1 : Prise en main

Exercice 1 : La mise en répertoire (Exo Hello)

Commençons doucement par créer un répertoire typique d'un projet Scala. Vous pouvez faire cette exercice pendant que vos téléchargements de Scala, l'IDE.

Cette exercice peut se faire intégralement avec nano/gedit !! Pour faire cette exercice, le SBT doit être impérativement installé (le reste peut se faire sans la liste plus haute).

```
mkdir hello
cd hello
mkdir -p src/{main,test}/{java,resources,scala}
mkdir lib project
```

- Nous allons créer notre premier fichier `Hello.scala`, pour ce faire, vous devrez créer un répertoire dans `src/main/scala/garden/bots`

Ensuite, nous allons définir un package nommé `garden.bots` suivi d'un Hello World

```
package garden.bots
```

```
object Hello extends App {
  println("Hello World!")
}
```

Remarque :

- Il n'y a pas de point virgule ! Si vous ajoutez un point virgule, vous serez alors en JavaScript.
- Nous avons perdu l'utilisation de `System.out` devant la méthode `println` !
- Nous ne pouvons pas encore compiler notre code car il manque l'équivalent d'un `pom.xml` de Maven. Nous allons donc devoir créer un fichier nommé `build.sbt` situé à la racine du projet (Donc au même endroit que le dossier `Hello`)

```
name := "01-Hello" // Nom de l'application
version := "1.0" // Version de l'application
ThisBuild / scalaVersion := "2.13.15" // Version de Scala à utiliser pour le projet actuel
```

- Lancez la commande `sbt run`
 - Vous verrez qu'il faut un certain temps à tout télécharger. En fait le `build.sbt` à l'heure actuelle c'est l'équivalent d'un `requirements.txt` sous python.
 - C'est aussi avec cette commande que vous pouvez lancer votre compilation et le run. Du coup vous devriez avoir le message Hello World qui devrait s'afficher.
 - Tips: Pour éviter d'attendre à chaque fois le lancement du `sbt run`, vous pouvez d'abord faire `sbt`, attendre et ensuite `run`. A chaque fois que vous souhaitez relancer la compilation et le run, vous pouvez faire juste `run`.

Nous allons modifier le programme pour introduire la notion de méthode.

```
object Hello {
  def main(args: Array[String]): Unit = {
    println("Hello World!")
  }
}
```

Remarque : Cela ressemble beaucoup à Java ! Le type de retour ici est `Unit`, c'est l'équivalent de `void` sous Java.

- Sous le terminal, allez directement où se trouve le `Hello.scala`
 - Lancez la commande `scalac Hello.scala` pour compiler. Vous devez être au même endroit que le fichier Scala pour que cela fonctionne.
 - Lancez la commande `scala Hello` pour lancer le programme.
- Nous allons maintenant créer des variables :
 - Une **var** : Associe un objet à un identificateur qui peut être modifié (mutable).
 - Une **val** : Associe un objet à un identificateur qui ne peut pas être modifié (non-mutable).

En Scala, nous allons privilégier les variable non-mutable !

- Créez un objet nommé `FrenchData` avec deux variables :
 - Le premier sera `now` qui récupère la date d'aujourd'hui à l'aide de la librairie `java.lang.LocalDate`
 - Le deuxième sera `df` : En utilisant

```
DateTimeFormatter.ofLocalizedDate(FormatStyle.LONG).withLocal(FRANCE)
```

- Importez les librairies suivantes :

```
import java.time.format.{DateTimeFormatter, FormatStyle}
import java.time.LocalDate
import java.util.Locale._
```

- Compilez le programme et lancez le code, que remarquez vous ?

Remarque :

- Sous Scala, il est possible d'importer plusieurs classes du même paquet. cf ligne `java.time.format` qui va importer les classe `DateTimeFormatter` et `FormatStyle`
- Pour importer l'ensemble des classes d'un paquets, nous utilisons `_` (cf `java.util.Locale._`)

Scala est un langage qui est totalement orienté objet, dans le sens où tout est un objet y compris les nombres.

Par exemple :

```
1 + 2 * 3 / x
```

est équivalent à

1. (2.*(3))./(x))

Ce qui nous amène à une remarque importante : le signe + ou - voir même * peuvent être utilisés en tant que nom de variable !

Les fonctions sous Scala sont également considéré comme des objets, qui est donc largement plus puissant que les expressions lambda sous Java.

Exercice 2 : Les fonction d'objets

Par rapport à Java, il y a très peu de différence entre les objets fonctions et les méthodes : vous pouvez passer des méthodes en tant qu'arguments, les stocker dans des variables et les renvoyer à partir d'autres fonctions, le tout sans syntaxe particulière. Cette capacité à manipuler les fonctions comme des valeurs est l'une des clés d'un paradigme de programmation très intéressant appelé programmation fonctionnelle, que vous avez vu en Haskell.

Nous allons créer un Objet Timer qui possédera trois fonction d'objet :

- `onePerSecond()`, qui va avoir comme paramètre d'entrée une callback, qui est lui même une fonction avec aucun retour (donc un `Unit` si vous avez suivi jusque là). Elle possédera une boucle `while` infinie, dont on appellera la méthode callback et on `sleep` le `Thread` pendant 1 seconde.

```
def oncePerSecond(callback: () => Unit): Unit = {
    while (true) { callback(); Thread.sleep(1000) }
}
```

Les `()` désigne une fonction, la flèche `=>` nous indique son type de retour.

- `timeFlies()` dont on va se contenter d'afficher un texte `time flies like an arrow...`
- une fonction `main()` dont le but sera tout simplement d'appeler `oncePerSecond()` ayant comme paramètre la méthode `timeFlies`.

Exécutez le programme, si tout se passe bien, normalement à chaque seconde, `time flies ...` s'affiche.

Fonction anonymes : Il existe des fonction anonyme sous Scala, c'est l'équivalent des lambda que vous connaissez sous Java. Nous pouvons donc simplifier le code de la manière suivante :

```
object Timer {
    def oncePerSecond(callback: () => Unit): Unit = {
        while (true) {
            callback()
            Thread.sleep(1000)
        }
    }

    def main(args: Array[String]): Unit = {
        // Ici nous avons une fonction anonyme.
        oncePerSecond(
            () => println("time flies like an arrow...")
        )
    }
}
```

L'avantage par rapport à Java : Nous n'avons pas besoin d'avoir une classe abstraite avec une fonction lambda vide.

- Testez le programme et vérifiez le que comportement du programme est identique au premier exemple.

Exercice 3 : Les classes

La création d'une classe est assez similaire à Java 16, c'est à dire les fameux Record.

```
class Complex(real: Double, imaginary: Double) {
  def re() = real // Notez qu'on est pas obligé de préciser le type, c'est inféré par le
  compilateur automatiquement ! Donc le compilateur sait déjà que c'est un type Double.
  def im() = imaginary
}
```

Cette classe Complex, possède un constructeur à deux paramètres définit entre les parenthèse. Elle possède deux getteurs qui retourne les valeurs de real et imaginary.

- Créer un programme ComplexNumber possédant un main qui crée un nombre complexe avec 1.2 en réel et 3.4 en imaginaire et affiche dans le terminal la partie imaginaire.
- Modifiez ce programme pour que les nombres soient données en paramètre au lancement du programme.

On souhaite répliquer le comportement des accès aux champs comme en Java ou en C à l'aide d'une structure. Autrement dit, nous ne souhaitons plus utiliser de parenthèse sur les attributs pour accéder aux champs de la classe.

Il est totalement possible de le faire en enlevant tout simple les parenthèse lors de la définitions des getteurs.

- Adaptez le code pour ne plus avoir de parenthèse dans les deux méthodes de getteurs (im et re).

Exercice 4 : Héritage et les overrides

A l'image de Java, dont toute les classes héritent automatiquement de Objects, il en va de même pour Scala. Toute classe de Scala héritent de scala.AnyRef.

Le fonctionnement reste le même par rapport à Java. A la différence que le mot clé override s'utilise d'une autre manière : on ne met pas le signe @ et que l'on met devant la définition d'une méthode.

Exemple :

```
override def equals(): Boolean = ???
```

- Redéfinir la méthode toString de l'exercice suivant afin que la sortie suivant soit affiché :

Nombre réel et de 1.2 ainsi que le nombre imaginaire est de 3.4

Exercice 5 : Type de données Algébriques, case class et Pattern Matchings

L'objectif de cette exercice est d'implanter une mini-calculatrice qui sera évalué sous la forme d'un arbre. Les expressions arithmétiques sera uniquement la somme par soucis de simplicité de la prise en main de Scala. Les exemples d'expression serait $1 + 2$ et $(x + x) + (7 + y)$.

Nous définissons

- un nœud comme étant une fonction (ici addition)
- une feuille comme étant une constante ou une variable.

```
abstract class Tree
object Tree {
  case class Sum(left: Tree, right: Tree) extends Tree // Un veritable record sous Scala !
  case class Var(n: String) extends Tree
  case class Const(v: Int) extends Tree
}
```

Maintenant que nous avons créé notre arbre syntaxique, nous allons commencer à définir les opérations pour les manipuler. Nous commençons par les évaluer dans une sorte d'environnement que l'on va définir tout la forme d'un type.

Exemple :

```
type Environment = String => Int
val ev: Environment = { case "x" => 5 }
```

Ces lignes doivent être défini dans un fichier appelé `Calc.scala`, ce qui donne :

```
import Tree._

object Calc {
  type Environment = String => Int
  val ev: Environment = { case "x" => 5 }

  def main(args: Array[String]): Unit = {

  }
}
```

Cette notation définit une fonction qui prends un certain x et renvoie la valeur 5. Les autres cas seront en échec.

Nous pouvons maintenant définir une fonction `eval` nous permettant d'utiliser à la fois notre arbre et notre environnement. Mais avant, nous devons savoir si notre valeur est une feuille, ou un nœud.

C'est à l'aide des pattern matching que l'on peut faire (qui est d'ailleurs très utilisé dans les pipelines customisés du traitement de la donnée).

```
def eval(t: Tree, ev: Environment): Int = t match {
  case Sum(left, right) => eval(left, ev) + eval(right, ev)
  case Var(n)           => ev(n)
  case Const(v)         => v
}
```

Pour ceux qui n'aurait pas compris :

1. il vérifie d'abord si l'arbre t est une `Sum`, et si c'est le cas, il lie le sous-arbre gauche à une nouvelle variable appelée `left` et le sous-arbre droit à une variable appelée `right`, et procède ensuite à l'évaluation de l'expression qui suit la flèche; cette expression peut utiliser (et utilise) les variables liées par le pattern apparaissant à gauche de la flèche, c'est-à-dire `left` et `right`,
2. si la première vérification échoue, c'est-à-dire si l'arbre n'est pas une `Sum`, il continue et vérifie si t est un `Var` ; si c'est le cas, il lie le nom contenu dans le nœud `Var` à une variable n et poursuit avec l'expression de droite,
3. si la deuxième vérification échoue également, c'est-à-dire si t n'est ni un `Sum` ni une `Var`, il vérifie s'il s'agit d'une `Const`, et si c'est le cas, il lie la valeur contenue dans le nœud `Const` à une variable v et passe à l'expression de droite,
4. enfin, si toutes les vérifications échouent, une exception est levée pour signaler l'échec de l'expression de correspondance de patterns ; cela ne pourrait se produire ici que si davantage de sous-classes d'arbre étaient déclarées.

- Écrire une fonction `main` qui vérifie que votre arbre fonctionne correctement.

Nous allons maintenant ajouter une nouvelle opération, mais qui ne sera pas basique : Une fonction dérivé !

```
import Tree._

def derive(t: Tree, v: String): Tree = t match {
  case Sum(left, right) => Sum(derive(left, v), derive(right, v))
  case Var(n) if v == n => Const(1)
  case _                => Const(0)
}
```

- Rédigez un programme `Main` de sorte à ce qu'on ai cette affichage

```
Expression: Sum(Sum(Var(x),Var(x)),Sum(Const(7),Var(y)))
Evaluation with x=5, y=7: 24
```

```

Derivative relative to x:
  Sum(Sum(Const(1),Const(1)),Sum(Const(0),Const(0)))
Derivative relative to y:
  Sum(Sum(Const(0),Const(0)),Sum(Const(0),Const(1)))

```

Exercice 6 : Les traits

Nous allons définir une notion de relation d'ordre sur les dates, pour cela, nous allons tout d'abord les représenter à l'aide des trait, possédant 4 méthodes ayant comme paramètre that de type Any, l'ensemble de ces méthodes renvoie un booléen :

- <
- <=
- >
- >=

Voici les première ligne de code

```

trait Ord {
  def < (that: Any): Boolean
  def <=(that: Any): Boolean = ???
  def > (that: Any): Boolean = ???
  def >=(that: Any): Boolean = ???
}

```

1. Compléter les trois méthodes restantes.

Maintenant, nous allons créer une héritage Date

```

class Date(y: Int, m: Int, d: Int) extends Ord {
  def year = y
  def month = m
  def day = d
  override def toString(): String = s"$year-$month-$day"

  // rest of implementation will go here
}

```

2. Redéfinissez la méthode equals pour qu'il compare deux dates et qu'ils soient égales.

Maintenant que nous avons notre equal, nous devons maintenant redéfinir les relations d'ordre

```

class Date(y: Int, m: Int, d: Int) extends Ord {
  // previous decls here

  def <(that: Any): Boolean = that match {
    case d: Date =>
      (year < d.year) ||
      (year == d.year && (month < d.month ||
        (month == d.month && day < d.day)))

    case _ => sys.error("cannot compare " + that + " and a Date")
  }
}

```

4. En vous aidant de l'exemple, redéfinissez les méthodes restante.

Partie 2 : HandsOn (Unix)

1. Clonez le répertoire suivant : <https://github.com/Noobzik/scala-class>
2. Lancer un terminal et saisissez la commande `handson go`

3. Ouvrez IntelliJ et placez vous sur le dossier `test src/test/scala/exercices/e00_scala_syntax.scala`

Partie 2 HandsOn (Windows)

1. Clonez le répertoire suivant à l'aide des commande de **git** : <https://github.com/Noobzik/scala-class/tree/048f7bca616e8230918d5c75dbdc558e0f96d2a3>
2. Tapez la commande suivante : `git checkout 048f7bca616e8230918d5c75dbdc558e0f96d2a3`
3. Saisissez la commande `./handson.bat go`
4. Ouvrez IntelliJ et placez vous sur le dossier `test src/test/scala/exercices/e00_scala_syntax.scala`